

AWS ML Study Guide

Concept notes, project references, and certification support

31 topics generated from the study catalog

00 Foundations

AWS EC2 (Elastic Compute Cloud)

00-foundations/ec2

Overview

EC2 gives you virtual machines on demand. You choose the instance family, operating system image, storage, networking, and scaling behavior, then AWS provides the compute capacity.

For ML teams, EC2 matters because it is the most flexible way to run custom training, inference, data processing, and supporting services when managed abstractions are too restrictive.

Core Concepts

- Instance families are optimized for different workloads such as general purpose, compute optimized, memory optimized, storage optimized, or GPU accelerated.
- An EC2 instance is usually launched from an AMI and attached to EBS volumes inside a VPC. Security groups, IAM roles, and bootstrap scripts define how it behaves after launch.
- Pricing options include on demand, reserved, and spot. Cost and interruption tolerance should influence how you choose between them for ML workloads.

How It Works

- You select an AMI, instance type, subnet, security group, IAM role, and storage layout. User data or configuration management then installs dependencies and starts services.
- If the instance hosts a long running service, it is usually placed behind a load balancer and managed by an Auto Scaling group for replacement and horizontal growth.
- If the instance hosts a one off training or data job, it can be provisioned, used, and terminated automatically as part of a workflow to avoid paying for idle compute.

AI and ML Relevance

- EC2 is common when you need custom CUDA drivers, niche libraries, background agents, or low level operating system access that is awkward in more managed services.
- GPU instances support deep learning training and high throughput inference, while CPU instances often handle preprocessing, orchestration, and lightweight model serving.

Operational Notes

- Patch management, kernel security, disk sizing, and monitoring are your responsibility on EC2. Flexibility comes with operational ownership.
- Use spot instances for fault tolerant training or batch processing, but combine them with checkpointing because the instance can be reclaimed.

- Right size aggressively. ML workloads can burn money quickly when overprovisioned GPUs or memory heavy instances are left running.

What To Remember

- EC2 is infrastructure, not a platform. You gain control and lose convenience.
- EBS stores block data for the instance; S3 stores durable object data outside the instance.
- For production systems, think in terms of replacement and automation, not logging into a single pet server.

AWS IAM (Identity and Access Management)

00-foundations/iam

Overview

IAM is the authorization layer that controls who can call AWS APIs and what they are allowed to do after they authenticate. It is the first security boundary for every workload you deploy in AWS.

In a machine learning environment, IAM protects datasets, training infrastructure, model artifacts, and deployment pipelines. If IAM is weak, the rest of the platform is weak no matter how good the model is.

Core Concepts

- IAM identities include users, groups, and roles. Human operators increasingly use federation or SSO, while workloads should use roles with temporary credentials instead of long lived access keys.
- Policies are JSON documents that define allowed or denied actions on resources, optionally constrained with conditions such as tags, source VPC endpoints, time, or MFA state.
- AWS evaluates identity based policies, resource policies, permission boundaries, session policies, and organization controls together. Explicit deny always overrides an allow.

How It Works

- A principal signs a request with credentials that represent a user or an assumed role. AWS then evaluates all applicable policies before the target service processes the request.
- Service roles are attached to workloads such as Lambda functions, EC2 instances, or SageMaker training jobs so the code can access S3, logs, and secrets without embedding credentials.
- Cross account access is commonly implemented by allowing one account to assume a role in another account. That is a standard pattern for promoting models from development to production.

AI and ML Relevance

- Training jobs often need read access to raw data, write access to model artifacts, and permission to emit logs and metrics. Those permissions should be separated from deployment permissions.
- A mature ML platform uses different roles for experimentation, scheduled pipelines, model registry operations, and production inference so blast radius stays small.

Operational Notes

- Prefer least privilege, short lived credentials, and role assumption over static keys. Long lived access keys in notebooks or scripts are a recurring source of compromise.
- Use CloudTrail and IAM Access Analyzer to review who used which permissions and where policies are broader than necessary.
- Separate human admin access from application access. People troubleshoot systems; workloads should have narrow machine oriented permissions.

What To Remember

- Authentication answers who you are. Authorization answers what you can do.
- Roles are the normal answer for AWS workloads. Users with keys should be the exception.
- If something is mysteriously denied, look for an explicit deny, a missing resource scope, or an organization level control.

AWS S3 (Simple Storage Service)

00-foundations/s3

Overview

S3 is AWS object storage. It stores data as objects inside buckets and is designed for very high durability, massive scale, and broad integration with the rest of AWS.

In ML systems, S3 is usually the default data lake and artifact store. Raw data, curated datasets, model files, pipeline outputs, logs, and static assets often live there.

Core Concepts

- Buckets are top level containers and objects are individual files or blobs addressed by key. Prefixes are naming conventions that simulate folders and support partitioned layouts.
- Important features include versioning, lifecycle rules, server side encryption, bucket policies, replication, event notifications, and multiple storage classes.
- S3 provides strong read after write and list consistency, which simplifies many ingestion and pipeline patterns compared with older eventual consistency assumptions.

How It Works

- Applications upload or download objects through the S3 API, SDKs, CLI, or signed URLs. IAM and bucket policies decide who can access which paths.
- Data engineering jobs commonly read from raw prefixes, transform the data, and write curated outputs in columnar formats such as Parquet for analytics and downstream ML use.
- Many services integrate directly with S3, including Athena, Glue, SageMaker, Lambda event notifications, and CloudTrail log delivery.

AI and ML Relevance

- Training datasets are often staged in S3, and model artifacts are written back there after the training job completes.
- S3 can trigger downstream workflows when new data lands, which is useful for automated feature computation, validation, batch inference, or retraining pipelines.

Operational Notes

- Design the bucket key structure deliberately. Bad partitioning and inconsistent naming make analytics, retention, and lifecycle management painful later.
- Encrypt sensitive data, control public access at both the bucket and account level, and use lifecycle rules to move cold data to cheaper storage classes.
- Do not treat S3 like a filesystem with heavy small file chatter when a compact columnar layout would be more efficient and cheaper.

What To Remember

- S3 is object storage, not block storage and not a database.
- Bucket policy plus IAM policy plus encryption controls together define access posture.
- For ML, think of S3 as the durable backbone for data and artifacts rather than just a place to drop files.

AWS VPC (Virtual Private Cloud)

00-foundations/vpc

Overview

A VPC is your logically isolated network in AWS. It defines the IP range, subnets, routing, and traffic boundaries for the compute and managed services you place inside it.

ML systems frequently handle sensitive data and expensive infrastructure, so network design matters. A good VPC layout reduces exposure, controls egress, and separates public entry points from private processing layers.

Core Concepts

- Subnets divide the VPC into smaller network zones, usually across multiple Availability Zones for resilience. Public subnets route to an internet gateway, while private subnets usually route outbound through a NAT or VPC endpoints.
- Security groups are stateful virtual firewalls attached to resources, while network ACLs are stateless controls applied at the subnet boundary.
- Route tables, VPC endpoints, peering, Transit Gateway, and DNS settings determine how traffic moves between services, accounts, and external networks.

How It Works

- You choose a CIDR range, create subnets, attach route tables, define security controls, and then launch resources into the appropriate subnets.
- Private workloads such as databases, internal APIs, or SageMaker jobs can access AWS services through VPC endpoints without traversing the public internet.
- Ingress is usually limited to controlled entry points such as an Application Load Balancer, API Gateway integration, or VPN connection from trusted environments.

AI and ML Relevance

- A common ML pattern is private training or inference infrastructure that reads data from S3 through endpoints and writes logs and metrics without needing public IP addresses.
- Network isolation is also important for compliance and for protecting proprietary models and feature data from accidental public exposure.

Operational Notes

- Plan CIDR blocks early. Renumbering a VPC later is painful, especially when multiple accounts or peered networks are involved.
- Use multiple Availability Zones for resilience, and capture VPC Flow Logs when network troubleshooting or auditability matters.
- Do not overexpose resources by putting everything in public subnets. Public accessibility should be the exception, not the default.

What To Remember

- Security groups normally express allowed flows better than broad subnet ACL rules.
- Public subnet does not mean the instance must be public. Internet reachability still depends on routing and public addressing.
- For secure ML platforms, private subnets plus endpoints are usually the baseline design.

01 ML Infrastructure

AWS ECR (Elastic Container Registry)

01-ml-infrastructure/ecr

Overview

ECR is AWS managed container image storage. It holds versioned Docker and OCI images that other services can pull when they need to start a container.

For ML teams, ECR is where custom training images, inference images, and utility containers are stored so that deployments are reproducible and centrally managed.

Core Concepts

- Repositories group related images, while tags and digests identify specific builds. Digests are immutable and better for strict reproducibility than floating tags.
- ECR supports vulnerability scanning, lifecycle policies, encryption, and repository policies for cross account sharing.
- Authentication is integrated with IAM. Build systems and runtimes usually obtain temporary credentials and then push or pull images through the ECR API.

How It Works

- A CI system builds the image, runs tests, tags it, and pushes it to an ECR repository.
- Services such as ECS, SageMaker, or Lambda container images then pull the image when creating or updating a workload.
- Multi account setups often replicate approved images from a build account into runtime accounts to separate software production from software consumption.

AI and ML Relevance

- ML inference containers often include model servers, preprocessing logic, and optimized native libraries. ECR ensures those dependencies are packaged consistently across environments.
- Training images in ECR let you standardize frameworks, CUDA libraries, and internal tooling for repeatable experiments and pipelines.

Operational Notes

- Avoid using mutable tags such as latest as the only deployment reference. Use digests or versioned tags in release automation.
- Enable lifecycle policies so old image layers do not accumulate forever and quietly increase storage cost.
- Scan images, patch base layers regularly, and keep the build chain deterministic so security and debugging stay tractable.

What To Remember

- ECR stores images; ECS, EKS, SageMaker, and Lambda run them.
- Tagging is for convenience. Digests are for certainty.
- Treat the container registry as part of the software supply chain, not just as storage.

AWS ECS (Elastic Container Service)

01-ml-infrastructure/ecs

Overview

ECS is AWS managed container orchestration. It schedules containers onto compute capacity and keeps services running according to the task definition you declare.

For ML applications, ECS is a practical choice for serving models, running supporting APIs, and executing recurring preprocessing or postprocessing jobs without managing Kubernetes.

Core Concepts

- Core objects include clusters, task definitions, tasks, and services. A task definition describes the containers, CPU, memory, networking, secrets, and startup behavior.
- ECS can run on EC2 instances that you manage or on Fargate, where AWS manages the underlying servers.
- Services maintain a desired number of healthy tasks and integrate with load balancers, service discovery, and deployment strategies.

How It Works

- You package the application as a container, store it in ECR, create a task definition, and then run it as a task or a service.
- For a long running inference API, ECS typically deploys tasks behind an Application Load Balancer and scales them based on CPU, memory, request count, or custom metrics.
- For scheduled or event driven jobs, ECS can launch short lived tasks that process a queue, transform data, or prepare artifacts for later training or inference stages.

AI and ML Relevance

- ECS works well for FastAPI model servers, embedding services, feature preprocessing workers, and internal tools that need container consistency but not the full weight of a larger platform.
- If your inference stack depends on custom native libraries or multiple sidecars, ECS often gives a cleaner operational model than forcing everything into a serverless boundary.

Operational Notes

- Right size task resources and container concurrency. Overcommitting CPU or memory leads to noisy failures and bad latency under load.
- Use IAM task roles, central logging, health checks, and rolling deployments so the service can update without downtime.
- Fargate reduces operational work, but EC2 backed ECS may be cheaper or more flexible for heavy GPU or host level customization requirements.

What To Remember

- Task definition is the runtime contract. Service is the controller that keeps tasks alive.
- Fargate optimizes convenience; EC2 launch type optimizes control.
- ECS is a strong middle ground when you want containers without adopting Kubernetes complexity.

AWS SageMaker

01-ml-infrastructure/sagemaker

Overview

SageMaker is AWS's managed machine learning platform for building, training, tuning, deploying, and governing ML models. It reduces the amount of infrastructure engineering needed to operationalize models.

Rather than giving you only raw compute, SageMaker gives purpose built ML abstractions such as training jobs, endpoints, pipelines, model registry integration, experiment tooling, and managed notebooks.

Core Concepts

- SageMaker separates major concerns into specific services such as Studio, training, processing, batch transform, endpoints, feature related tooling, and pipeline orchestration.
- A typical SageMaker workflow uses S3 for data and artifacts, IAM for execution roles, CloudWatch for logs and metrics, and VPC integration when private networking is required.
- The value of SageMaker is not only managed infrastructure. It also standardizes packaging, deployment, lineage, and repeatability across teams.

How It Works

- You prepare data, choose or build a container image, define the training configuration, and launch a managed job that provisions compute only for the duration needed.
- The trained model artifact can then be registered, deployed to a real time endpoint, served asynchronously, used for batch transform, or promoted through an MLOps pipeline.
- Operational events such as logs, metrics, model approval, and endpoint scaling are integrated into the wider AWS control plane.

AI and ML Relevance

- SageMaker is useful when you want managed distributed training, hyperparameter tuning, controlled deployment patterns, or an internal ML platform without building every layer from scratch.
- It is especially strong for teams that need repeatable ML workflows with governance, auditability, and tighter integration with AWS-native security and networking patterns.

Operational Notes

- SageMaker reduces toil but does not remove design work. You still need disciplined data versioning, evaluation gates, cost controls, and clear ownership boundaries.
- Use VPC attachment, KMS encryption, and separate execution roles when the platform handles regulated or proprietary data.
- Choose SageMaker because the managed abstractions help, not because it is fashionable. Simple workloads may be cheaper elsewhere, while complex governed ML often benefits from it strongly.

What To Remember

- SageMaker is a platform layer built on top of core AWS services, not a separate universe.
- Training, deployment, and pipeline concerns should be understood independently even if SageMaker can connect them.
- For exams and interviews, focus on when SageMaker reduces operational burden versus when EC2 or containers give needed control.

AWS SageMaker Deployment

01-ml-infrastructure/sagemaker/deployment

Overview

SageMaker Deployment turns trained models into managed inference endpoints or batch inference jobs. It packages model artifacts, container logic, scaling behavior, and network settings into a serving configuration.

The value is operational consistency. Teams can promote models through controlled environments instead of treating deployment as a manual afterthought.

Core Concepts

- A deployment usually starts with a model definition that points to the serving image and the model artifact in S3.
- Endpoint configurations describe instance types, variant weights, autoscaling behavior, or specialized modes such as asynchronous or serverless inference.
- SageMaker supports patterns such as multi model endpoints, blue green style updates through variants, and data capture for later monitoring.

How It Works

- After a model is approved, the deployment system creates or updates the endpoint configuration and then rolls traffic to the new endpoint variant.
- Clients send inference requests to the hosted endpoint, while logs, metrics, and optionally request payload capture flow to surrounding AWS services.
- For offline scoring, the same model package can be used in batch transform rather than a permanently running real time endpoint.

AI and ML Relevance

- Deployment design affects latency, throughput, cost, and release safety. The right mode depends on traffic profile, payload size, and how quickly results are needed.
- ML teams also care about safe promotion, shadowing, rollback, and post deployment quality checks because correctness drift can be more subtle than simple uptime failures.

Operational Notes

- Use autoscaling carefully. If cold starts or model load time are high, a low minimum instance count can cause unstable latency.
- Capture inference data where governance allows it so you can debug failure cases, audit outcomes, and build monitoring or retraining signals.
- Separate release approval from endpoint mutation. Production deployment should be traceable to a specific model version and image version.

What To Remember

- Training creates a model artifact; deployment turns it into a callable service contract.
- Real time, asynchronous, serverless, and batch inference exist for different traffic and cost profiles.
- The hardest part is rarely the API call. It is the release discipline around versions, rollbacks, and observability.

AWS SageMaker Training

01-ml-infrastructure/sagemaker/training

Overview

SageMaker Training runs managed training jobs on ephemeral infrastructure. You provide code, data locations, and resource configuration, and SageMaker provisions the instances, streams logs, stores artifacts, and tears the instances down when the job completes.

This is the core managed primitive for repeatable training because it separates model code from the underlying host lifecycle and makes experiments easier to automate.

Core Concepts

- A training job is defined by the training image, entry point script, input channels, output path, instance count, instance type, and optional distributed training settings.
- Data can be delivered through multiple modes such as file, fast file, or pipe depending on access patterns and startup needs.
- Managed spot training, checkpoints, profiler tooling, and metrics extraction can reduce cost and improve visibility for long or expensive jobs.

How It Works

- The training script is packaged with framework specific estimators or custom containers and launched with a role that can access the required data and artifact locations.
- SageMaker downloads or streams the data, executes the container, captures logs to CloudWatch, and writes the model artifact and any generated metrics back to S3.
- A completed job can feed later stages such as model evaluation, registration, batch transform, or endpoint deployment.

AI and ML Relevance

- Managed training is well suited for scheduled retraining, hyperparameter tuning, and experiments that need clean, isolated, reproducible execution environments.
- Distributed training support matters when model size or dataset size outgrows a single node, especially for deep learning workloads on GPU fleets.

Operational Notes

- Checkpointing is essential when using spot instances or any training run that would be expensive to restart from zero.
- Track both infrastructure metrics and model metrics. Good GPU utilization with bad validation loss still means the experiment failed.
- Keep containers lean and training inputs explicit so runs are reproducible and startup times do not become an invisible source of waste.

What To Remember

- Training jobs are ephemeral by design. Persistent state belongs in S3 or attached managed storage, not only inside the container filesystem.
- Input channels define what the job sees; artifact output defines what the rest of the platform can reuse.
- If you cannot reproduce a training run from code, data version, parameters, and image version, the platform is not mature enough.

02 MLOps Pipelines

AWS CodePipeline

02-mlops-pipelines/codepipeline

Overview

CodePipeline is AWS's managed continuous delivery orchestrator. It connects source, build, test, approval, and deployment actions into a stage based release flow.

Its main value is disciplined automation. Instead of relying on a person to remember deployment steps, the pipeline executes the same controlled sequence every time.

Core Concepts

- A pipeline is composed of stages, and each stage contains actions such as source retrieval, CodeBuild execution, manual approval, or deployment integration.
- Artifacts move from one stage to the next so that the exact built output is what gets tested and deployed.
- Pipelines integrate well with IAM, KMS, EventBridge, and downstream deployment targets such as ECS, Lambda, CloudFormation, or custom actions.

How It Works

- A code or configuration change triggers the pipeline, which pulls the source and passes it into automated build and validation steps.
- Successful outputs are packaged as artifacts, optionally gated by manual approval, then deployed to the target environment through a controlled action.
- Notifications and audit trails make it clear which version moved where and whether a failure happened in source, build, test, or deployment.

AI and ML Relevance

- In ML systems, CodePipeline is often used for infrastructure as code, deployment of inference services, packaging of training images, or promotion of approved model packages.
- It is particularly useful when the broader platform standardizes on AWS native CI and CD rather than an external orchestration tool.

Operational Notes

- Keep pipeline stages purposeful. Mixing unrelated validation and deployment logic into one opaque step makes failures harder to debug.
- Store artifacts securely and make the pipeline idempotent so reruns do not create confusing partial state.
- Use manual approval only where human judgment adds real value. Too many approval steps turn automation into ceremony.

What To Remember

- CodePipeline is a release conveyor belt. It is most valuable when release steps are standardized and auditable.
- Artifacts matter because they preserve what was actually built and tested.
- It solves software promotion, not experiment tracking. That is an important boundary in ML systems.

GitHub Actions with AWS

02-mlops-pipelines/github-actions-aws

Overview

GitHub Actions is a CI and automation system that runs directly from a GitHub repository. When paired with AWS, it becomes a common control plane for testing, packaging, and deploying cloud workloads from version controlled changes.

For many teams, GitHub Actions is attractive because code review, merge policy, and build automation all live in the same developer workflow.

Core Concepts

- Workflows are defined in YAML and contain jobs and steps that run on GitHub hosted or self hosted runners.
- The most secure AWS integration pattern is usually OpenID Connect role assumption, which lets GitHub obtain short lived AWS credentials without storing static secrets.
- Reusable workflows, environment protections, artifact passing, and matrix execution help scale the setup across multiple services or model variants.

How It Works

- A pull request or push event triggers the workflow, which then runs tests, linters, packaging steps, and security checks.
- If deployment is allowed, the workflow assumes an AWS role and performs actions such as pushing an image to ECR, updating ECS, invoking a SageMaker pipeline, or applying infrastructure changes.
- Workflow status feeds back into the repository so release readiness is visible during code review rather than after a merge.

AI and ML Relevance

- GitHub Actions is often used to build training and inference containers, validate ML API code, trigger retraining workflows, or gate model deployment on evaluation outputs.
- It also helps when the ML platform spans more than AWS alone, because GitHub can coordinate multi system automation from one source change event.

Operational Notes

- Pin third party actions to trusted versions, avoid overprivileged tokens, and prefer OIDC over long lived AWS keys.
- Use environment protections and branch rules so production credentials are reachable only from reviewed code paths.
- Do not overload one workflow with every responsibility. Smaller purpose built workflows are easier to secure and troubleshoot.

What To Remember

- GitHub Actions owns repository centered automation; AWS owns the runtime resources.
- OIDC role assumption is the modern security baseline for GitHub to AWS federation.
- For ML teams, the tool is most useful when connected to clear quality gates rather than only to deployment mechanics.

MLflow on AWS

02-mlops-pipelines/mlflow-on-aws

Overview

MLflow provides experiment tracking, model registry style capabilities, and artifact management for machine learning work. Running it on AWS makes it accessible to a team while using AWS storage, databases, and security controls.

Its real value is organizational memory. It records what ran, with which parameters, on which data, and with what results so model development becomes auditable and comparable.

Core Concepts

- A typical MLflow setup includes a tracking server, a backend store for metadata, and an artifact store such as S3 for model files and plots.
- Runs capture parameters, metrics, tags, and artifacts. Registered models organize versioned outputs that teams may later evaluate or deploy.
- MLflow does not replace the entire production platform. It complements it by handling experiment lineage and model packaging at the experimentation layer.

How It Works

- A notebook, script, or training job logs metrics and artifacts to the tracking server while the run is executing.
- Team members compare runs, inspect outputs, and decide which model versions deserve promotion, registration, or deeper evaluation.
- When integrated with an MLOps pipeline, an approved run can trigger packaging, validation, or deployment workflows in AWS.

AI and ML Relevance

- MLflow is useful when multiple experiments, datasets, and parameter combinations are being tested and human memory is no longer reliable enough to track what worked.
- It also provides a bridge between ad hoc experimentation and operational model release by giving each promising artifact a stable identity and metadata trail.

Operational Notes

- Protect the tracking server with authentication, network isolation, and backups for the metadata store. Experiment history is operationally important.
- Separate artifact storage from code repositories. Model outputs can be large and need lifecycle and retention management.
- Decide early which metadata fields are mandatory, or the run history will become inconsistent and hard to search.

What To Remember

- MLflow answers what happened during experimentation; deployment systems answer what is live in production.
- Without disciplined logging, MLflow becomes a dump of half labeled runs. Governance needs conventions.
- For teams iterating quickly, experiment tracking is not optional once more than one person or one model is involved.

AWS SageMaker Pipelines

02-mlops-pipelines/sagemaker-pipelines

Overview

SageMaker Pipelines is a managed workflow service for machine learning lifecycle orchestration inside SageMaker. It expresses ML steps as a directed graph with explicit inputs, outputs, parameters, and dependencies.

Unlike generic CI pipelines, it is designed around ML concerns such as processing, training, evaluation, conditional registration, and repeatable data driven execution.

Core Concepts

- Pipelines can include processing steps, training steps, tuning jobs, evaluation logic, conditional branches, cache reuse, and model registration.
- Pipeline parameters let you vary dataset paths, instance types, thresholds, or model package groups without rewriting the pipeline definition.
- Step outputs become named references for downstream stages, which improves reproducibility and makes the workflow graph easier to reason about.

How It Works

- A pipeline definition is created in code and submitted to SageMaker, where it becomes a reusable managed workflow.
- When the pipeline runs, preprocessing produces features, training creates a model artifact, evaluation computes quality metrics, and a conditional step decides whether the model should be registered or deployed.
- Each run leaves an execution record, which helps trace exactly which data, parameters, and images were involved in a given model version.

AI and ML Relevance

- This service is useful when retraining or promotion needs to happen repeatedly with discipline, especially across multiple environments or business units.
- It aligns well with model governance because quality thresholds, approval gates, and lineage are represented as part of the workflow rather than as informal side notes.

Operational Notes

- Keep the pipeline definition in version control and treat it like application code. Hidden notebook logic is a common source of unreproducible ML systems.
- Use caching and incremental design where possible, but ensure cache reuse does not mask a legitimate need to rerun a step after data or code changes.
- Be explicit about what constitutes a deployable model. A finished training job is not automatically a production ready outcome.

What To Remember

- SageMaker Pipelines is about repeatable ML execution, not general source code CI.
- The core mental model is preprocess, train, evaluate, decide, and promote.
- A high quality pipeline makes model release criteria visible and enforceable instead of tribal knowledge.

03 Serverless

AWS API Gateway

03-serverless/api-gateway

Overview

API Gateway is a managed front door for HTTP, REST, and WebSocket APIs. It receives client traffic, applies policies, and forwards requests to backend integrations.

For ML applications, API Gateway often sits in front of inference services, orchestration Lambdas, or thin application layers that expose model capabilities to external consumers.

Core Concepts

- Core features include routing, request validation, throttling, authorization, custom domains, stage management, and integration mapping.
- Backends can be Lambda functions, HTTP services, private integrations, or other AWS services depending on the API design.
- Usage plans, API keys, authorizers, and logging help separate public access patterns from internal or partner access patterns.

How It Works

- A client sends a request to the API endpoint, API Gateway applies authentication and request handling rules, then forwards the request to the configured integration.
- The backend returns a response, which API Gateway can transform, enrich with headers, or standardize before returning to the caller.
- Operational features such as throttling and stage based rollout help protect backends and manage gradual release of new behavior.

AI and ML Relevance

- ML APIs benefit from API Gateway because it centralizes security, quotas, and routing policies before traffic reaches expensive inference components.
- It is also a clean way to expose asynchronous prediction patterns or file based workflows where the initial request starts work and later calls fetch results.

Operational Notes

- Be aware of payload sizes, timeout limits, and latency overhead. Not every inference pattern fits cleanly through a synchronous API boundary.
- Use structured request validation and authorization rather than trusting each backend to implement every guard consistently.

- Logging and correlation identifiers matter. When an inference path spans API Gateway, Lambda, and a model backend, debugging needs end to end traceability.

What To Remember

- API Gateway is the contract boundary, not the business logic itself.
- It protects and shapes backend access; it does not replace backend capacity planning.
- In ML systems, think about cost per request and latency amplification before exposing a heavy model behind a public API.

AWS Lambda

03-serverless/lambda

Overview

Lambda is AWS's event driven serverless compute service. You upload code or a container image, define triggers, and AWS runs the function when events arrive.

Its strength is removing server management for short lived tasks. It works best when the unit of work is discrete, stateless, and can tolerate cold start behavior within the required latency envelope.

Core Concepts

- A Lambda function has a handler, runtime, memory allocation, timeout, execution role, and optionally layers or a container image package.
- Triggers can come from API Gateway, S3 events, EventBridge, queues, streams, Step Functions, and many other AWS sources.
- Concurrency settings, retries, dead letter handling, and event source mappings shape how the function behaves under load or failure.

How It Works

- An event arrives, AWS allocates or reuses an execution environment, and the handler runs with temporary credentials from the function role.
- The function typically reads payload data, calls AWS services or external APIs, and returns a response or writes outputs to storage or queues.
- If traffic spikes, Lambda scales horizontally by creating more execution environments until quotas or downstream limits are reached.

AI and ML Relevance

- Lambda is useful for data validation, feature preprocessing, lightweight inference, orchestration glue, model trigger logic, and event driven postprocessing around a larger ML system.
- It is also a strong companion to S3 driven workflows, where object arrival triggers enrichment or dispatch into downstream training or scoring pipelines.

Operational Notes

- Package size, startup time, and dependency bloat directly affect cold starts. Heavy ML frameworks can make Lambda a poor fit for synchronous inference.
- Tune memory and timeout based on actual runtime behavior. More memory can increase CPU allocation and reduce execution time enough to improve both latency and cost.
- Protect downstream services with queues, reserved concurrency, or backpressure aware designs. Lambda can scale faster than the systems it calls.

What To Remember

- Lambda is excellent glue code, but it is not magic. Some workloads still need containers or managed endpoints.
- Cold starts matter most when latency is user facing and the runtime package is large.
- Think event contract first, function code second. Good serverless systems are driven by clear events and failure handling.

04 Data Engineering

AWS Athena

04-data-engineering/athena

Overview

Athena is a serverless interactive query service for data stored in S3. It lets you run SQL over files without provisioning or managing a dedicated analytics cluster.

It is valuable in ML workflows because teams need a fast way to inspect datasets, validate assumptions, sample training windows, and investigate predictions without building separate infrastructure first.

Core Concepts

- Athena uses schema on read, usually through the Glue Data Catalog, which means the meaning of the data is defined at query time rather than baked into a database load step.
- Performance and cost depend heavily on file format, compression, partitioning, and how much data each query scans.
- Workgroups, query result locations, and access controls help separate exploratory use from governed production style use.

How It Works

- Data is stored in S3, a table definition describes where it lives and how it is structured, and Athena executes SQL against that definition using an underlying distributed engine.
- Query results are written back to S3 and can be inspected directly, consumed by notebooks, or used to feed later data processing or reporting steps.
- Athena is often used iteratively: explore raw data, refine filters, validate counts, and then codify the proven logic in a more formal pipeline if it becomes recurring.

AI and ML Relevance

- Athena is useful for exploring labeling quality, checking class imbalance, auditing inference logs, and quickly slicing large event datasets before training decisions are made.
- It also complements Glue well, because cataloged curated datasets become directly queryable for both analysts and ML engineers.

Operational Notes

- Small files and poor partitions make Athena expensive and slow. File layout is not an implementation detail; it is part of performance design.
- Prefer columnar formats like Parquet for analytical workloads so only relevant columns are scanned.
- Athena is great for exploration and light operational analytics, but not a replacement for every transactional or low latency query requirement.

What To Remember

- Serverless SQL is convenient, but the data still needs structure and discipline.
- You pay largely for scanned data, so data layout drives cost.
- Athena is often the shortest path from raw S3 data to a useful answer during ML investigation.

AWS DynamoDB

04-data-engineering/dynamodb

Overview

DynamoDB is AWS's managed key value and document database. It is designed for high scale, predictable low latency access patterns, and minimal operational overhead.

In ML systems, DynamoDB is often useful at the serving edge where request latency matters and the access pattern is known ahead of time.

Core Concepts

- The primary design unit is the table with a partition key and optional sort key. Secondary indexes support alternate query patterns, but they should be designed intentionally.
- Capacity can be provisioned or on demand, and features such as TTL, streams, and transactions extend the base storage model for event driven and operational use cases.
- Data modeling starts from access patterns, not from trying to normalize relationships the way you would in a relational schema.

How It Works

- An application writes or reads items by key, query, or scan, though scans should be treated cautiously because they are rarely the right production access pattern.
- Streams can publish item changes to downstream systems, which is useful for event driven enrichment or cache invalidation flows.
- Global tables or replication patterns can support multi region designs when low latency or resilience requirements justify the complexity.

AI and ML Relevance

- DynamoDB is a strong fit for feature lookups, prediction caches, idempotency records, session state, or metadata that must be served quickly during inference.
- It is also useful when upstream ML results need to be materialized into an application friendly structure for user facing workloads.

Operational Notes

- Hot partitions are a common failure mode. If the partition key concentrates too much traffic, the table can suffer throttling even when the aggregate capacity seems sufficient.
- Model the access pattern first and test it under realistic request skew. A clean schema on paper is not enough if runtime traffic clusters badly.
- Use scans sparingly, watch item sizes, and understand consistency choices so performance behavior is deliberate rather than accidental.

What To Remember

- DynamoDB rewards access pattern driven design and punishes generic relational instincts.
- Low latency comes from the right key design, not from magical auto scaling alone.
- For inference systems, it is often the supporting state store, not the place where large training datasets belong.

AWS Glue

04-data-engineering/glue

Overview

Glue is AWS's serverless data integration service. It combines a data catalog, schema discovery, and ETL style jobs for moving and transforming data across analytical pipelines.

For ML, Glue is often the bridge between raw operational data and curated training or analytics datasets that downstream systems can trust.

Core Concepts

- The Glue Data Catalog stores table definitions and partitions so services like Athena and Spark jobs can agree on dataset structure.
- Crawlers inspect data sources and infer schemas, though mature teams often override or curate those schemas rather than trusting inferred output blindly.
- Glue jobs can run Spark based transformations at scale, with job bookmarks to track progress across repeated incremental runs.

How It Works

- Data lands in S3 or another source system, a crawler or manual schema registration defines metadata, and a Glue job transforms the data into a cleaned output location.
- The curated data is then queryable through Athena, consumable by SageMaker, or passed into downstream reporting and feature engineering systems.
- Schedules or event driven triggers let the pipeline refresh as new source data arrives.

AI and ML Relevance

- Glue helps standardize training data preparation, feature extraction jobs, and schema management for large datasets that exceed what ad hoc notebook code should be trusted to handle.
- It is especially useful when multiple consumers need the same curated datasets and you want transformation logic to live in a managed, repeatable pipeline.

Operational Notes

- Schema drift is common. If upstream producers change field names or types, downstream jobs can silently degrade unless validation is explicit.
- Partition data and use efficient file formats such as Parquet so jobs and queries do not waste time scanning unnecessary data.
- Glue is powerful, but not every transformation needs Spark. Use simpler tools for small, local, or latency sensitive tasks.

What To Remember

- Think of Glue as both metadata infrastructure and transformation infrastructure.
- The catalog is often just as important as the ETL job because discoverability and shared schema matter across teams.
- Reliable ML depends on reliable data preparation, and Glue is one way to formalize that layer.

AWS RDS (Relational Database Service)

04-data-engineering/rds

Overview

RDS is AWS managed relational database infrastructure for engines such as PostgreSQL, MySQL, MariaDB, SQL Server, Oracle, and Aurora variants. It handles much of the undifferentiated database operations work while preserving the relational model.

Relational systems remain important in ML platforms because not every dataset is a data lake object. Metadata, transactional application data, user records, and workflow state often fit better in a relational store.

Core Concepts

- RDS manages backups, patching windows, storage, failover features, and monitoring integrations, while the database engine still enforces schemas, indexes, and SQL semantics.
- Important design dimensions include engine choice, Multi Availability Zone deployment, read replicas, storage performance, and connection management.
- Aurora is related but distinct. It offers a cloud optimized relational engine with different performance and scaling characteristics compared with standard RDS engines.

How It Works

- You provision the database into subnets within a VPC, secure it with security groups, and connect applications through standard database drivers.
- Applications issue SQL queries or transactions, while RDS handles storage, backups, failover settings, and many operational hooks in the background.
- Read replicas or analytics exports can separate operational read pressure from write heavy transactional workloads.

AI and ML Relevance

- In ML environments, RDS often stores application ground truth, labeling workflow state, feature metadata, audit records, or model deployment metadata that benefits from strong consistency and joins.
- It can also act as an operational system of record whose data is later extracted into S3 and transformed for training or reporting.

Operational Notes

- Manage schema changes deliberately. Database migrations are production changes and should be treated with the same care as application deployment.
- Watch connection counts carefully when serverless or autoscaling clients are involved. Connection storms are a classic failure mode with relational backends.
- Choose RDS for relational access patterns, not as a default container for any data just because SQL is familiar.

What To Remember

- RDS is about managed operations around relational databases, not a new data model.
- Strong consistency and relational joins are the main reasons to use it in ML systems.
- If access patterns are simple key based lookups at very high scale, DynamoDB may be a better fit than forcing the problem into SQL.

05 Monitoring

AWS CloudTrail

05-monitoring/cloudtrail

Overview

CloudTrail records AWS API activity. It is the audit log of who called what, from where, and when across your AWS accounts.

This matters in ML platforms because operational and security incidents often revolve around control plane changes: a role was broadened, a training job was launched, an endpoint was deleted, or a bucket policy changed.

Core Concepts

- Management events cover control plane actions such as creating resources or changing permissions. Data events cover high volume object or item level activity for selected services such as S3.
- Trails can be account scoped or organization wide, and their logs are usually delivered to S3 with optional integration into analytics or alerting systems.
- CloudTrail Lake and event history features provide additional ways to search and analyze activity, but the core value remains reliable auditability.

How It Works

- A user, role, or service calls an AWS API. CloudTrail records the event metadata and writes it into the configured delivery and retention path.
- Security teams or operators query those logs to answer investigative questions such as who changed this policy, which role deleted the endpoint, or when access started failing.
- CloudTrail records can also drive detections when sensitive actions occur outside expected automation paths.

AI and ML Relevance

- CloudTrail helps prove whether a model deployment was automated, whether a dataset bucket was accessed unexpectedly, or whether a training cluster was launched by a human rather than by the approved pipeline.
- It is also useful for governance because regulated ML workloads need change history around access, release, and data handling operations.

Operational Notes

- Enable organization wide coverage where possible, and protect the destination logs from tampering with strong access controls and retention design.
- Data events can be high volume and higher cost, so enable them where their investigative value is worth the overhead.

- CloudTrail is most useful before an incident, not after. Missing audit coverage cannot be reconstructed retroactively.

What To Remember

- CloudWatch tells you a system is unhealthy; CloudTrail often tells you what changed.
- It is an audit and governance service, not a performance monitoring tool.
- In mature ML environments, every production change path should be traceable through CloudTrail records.

AWS CloudWatch

05-monitoring/cloudwatch

Overview

CloudWatch is AWS's monitoring and observability platform for metrics, logs, alarms, dashboards, and event driven reactions. It is where many AWS workloads publish the signals needed to understand health and performance.

For ML systems, observability is not limited to uptime. You also need visibility into latency, throughput, cost behavior, data freshness, and signals that may hint at model quality issues.

Core Concepts

- Metrics represent numeric time series such as CPU load, request count, endpoint latency, or custom business measurements.
- Logs capture detailed execution records from services such as Lambda, ECS, and SageMaker training or inference jobs.
- Alarms, anomaly detection, dashboards, metric math, and event routing let operators turn raw telemetry into operational action.

How It Works

- Workloads emit logs and metrics automatically or through custom instrumentation. CloudWatch stores and indexes those signals.
- Operators define dashboards and alarms that react to thresholds, rate changes, missing heartbeats, or composite conditions.
- Triggered alarms can notify humans, start automations, or feed incident workflows through SNS, EventBridge, or other integrations.

AI and ML Relevance

- For ML, CloudWatch tracks endpoint latency, error rate, queue depth, training job duration, GPU utilization proxies, and other indicators that affect both customer experience and cost.
- Custom metrics can also represent domain signals such as low confidence prediction counts, fallback rate, or the volume of data rejected by validation logic.

Operational Notes

- Choose metrics intentionally. High cardinality dimensions and noisy custom metrics can create cost and usability problems.
- Logs need retention policies and parsing conventions, or they become expensive text dumps that nobody can search effectively during an incident.
- Monitoring does not equal understanding model quality, but it provides the operational surface where model issues first become visible in production.

What To Remember

- Metrics answer how much and how often; logs answer what happened in detail.
- Alarms should be actionable. If a page fires and nobody knows what to do, the signal design is weak.
- For ML systems, combine infrastructure telemetry with business and model oriented telemetry to avoid blind spots.

06 GenAI

AWS Bedrock

06-genai/bedrock

Overview

Bedrock is AWS's managed generative AI platform for working with foundation models through API based access without managing the underlying model serving infrastructure yourself.

It is useful when you want to build generative AI features on AWS while focusing on prompt design, retrieval, governance, and application behavior rather than GPU fleet operations.

Core Concepts

- Bedrock provides access to multiple model families through a unified style of managed integration, along with features for guardrails, knowledge based augmentation, and agent oriented workflows.
- The platform fits well with AWS identity, network, and observability controls, which can matter for enterprises that need private governance around generative AI usage.
- The key abstraction is managed model invocation. You work with prompts, context, configurations, and outputs rather than building the serving stack yourself.

How It Works

- An application sends a request containing the prompt and any supporting context to the chosen model endpoint through Bedrock's API.
- Optional retrieval, policy, or orchestration layers can enrich the request before model inference, and the output is then returned to the application for postprocessing or user presentation.
- Teams can wrap this with evaluation, prompt versioning, caching, and safety controls to create a governed production path.

AI and ML Relevance

- Bedrock is relevant when ML teams need chat, summarization, extraction, or retrieval augmented generation features but do not want to self host large models.
- It also enables a practical split of responsibilities: platform teams manage security and cost guardrails, while product teams iterate on prompts, context, and UX behavior.

Operational Notes

- Do not confuse easy API access with solved product quality. Prompt design, retrieval quality, hallucination mitigation, and evaluation discipline still matter.
- Watch token usage and latency, because generative AI cost scales differently from traditional endpoint hosting.
- Treat private data carefully. Context sent to a model is still a data handling event and must respect governance boundaries.

What To Remember

- Bedrock abstracts model hosting, not application design risk.
- Retrieval quality and evaluation quality often matter more than swapping from one model to another.
- For production genAI, governance and observability are as important as raw model capability.

AWS SageMaker JumpStart

06-genai/sagemaker-jumpstart

Overview

SageMaker JumpStart provides curated solution templates, prebuilt models, and starter workflows inside the SageMaker ecosystem. It is designed to accelerate the path from interest to a working baseline.

Instead of building every experiment from zero, teams can use JumpStart to evaluate existing models, deploy quick prototypes, or fine tune supported foundations before investing in heavier custom engineering.

Core Concepts

- JumpStart exposes ready made assets such as notebooks, model cards, deployment templates, and fine tuning workflows depending on the model family.
- It sits inside the broader SageMaker platform, which means the resulting artifacts can still participate in SageMaker training, hosting, and operational workflows.
- The main abstraction is acceleration through curated starting points rather than invention of a new runtime model.

How It Works

- A team selects a model or solution, reviews the configuration, launches the baseline environment, and tests whether the pretrained capability or starter architecture fits the use case.
- If needed, the model is fine tuned or adapted with domain data and then deployed through the normal SageMaker serving options.
- Once validated, the prototype can be folded into the same governance and CI or CD processes used for other managed ML assets.

AI and ML Relevance

- JumpStart is valuable when you need to move quickly from idea to benchmark and want a supported path for exploring pretrained or foundation model options on AWS.
- It reduces the amount of boilerplate engineering needed to answer the first question: does this model family solve enough of the problem to justify deeper work?

Operational Notes

- Curated does not mean production ready by default. Review licensing, instance cost, security posture, and quality before promoting a JumpStart based prototype.
- Generated assets are starting points. Teams should still harden monitoring, release controls, and data handling behavior before using them in critical environments.
- Beware of false confidence from a quick demo. Baseline success must still be tested against real data, constraints, and cost targets.

What To Remember

- JumpStart shortens initial exploration, but it does not replace platform judgment.
- The fastest path to a demo is not always the safest path to production.
- It is best used to accelerate evaluation, prototyping, and informed model selection.

07 Projects

Project: Batch Inference on S3

07-projects/batch-inference-s3

Overview

This project pattern scores large datasets asynchronously by reading inputs from S3, running offline inference, and writing prediction outputs back to S3. It is the right shape when throughput matters more than immediate per request latency.

Batch inference is common for nightly recommendations, fraud review backlogs, document processing, or backfilling predictions across historical data.

Core Concepts

- The architecture typically includes S3 for input and output, an orchestration layer such as Step Functions or a scheduled pipeline, compute for preprocessing and inference, and monitoring for job status and quality.
- The inference engine may be SageMaker Batch Transform, a custom container job on ECS, or another managed compute path depending on the model and scaling requirements.
- Partitioning, manifest generation, idempotency, and result versioning are first class concerns because batch jobs are large and often retried.

How It Works

- Raw input files land in S3 and are validated, partitioned, or converted into the format expected by the model runtime.
- The batch job reads the input, performs inference at scale, writes structured prediction outputs to a new S3 prefix, and emits completion metrics and error summaries.
- Downstream consumers then query or load the results into operational systems, dashboards, or further analytics pipelines.

AI and ML Relevance

- This pattern is attractive when model initialization is heavy, data volumes are large, and the business can accept delayed delivery of results.
- It also creates a clean audit trail because both inputs and outputs are stored durably, versioned, and reprocessable if the model or logic changes.

Operational Notes

- Design for retries and partial failures. Large batch jobs fail in the middle more often than teams expect.
- Store run metadata such as input version, model version, code version, and thresholds so you can explain later why a given output set looks the way it does.
- Optimize file sizes and parallelism. Too few giant files or too many tiny files can both hurt throughput and cost.

What To Remember

- Batch inference is an offline factory, not an online API.
- Version every run so predictions remain traceable to their exact inputs and model.
- When data volume is high and latency is flexible, this pattern is often cheaper and operationally safer than always on real time serving.

Project: Model Deployment Pipeline

07-projects/model-deployment-pipeline

Overview

A model deployment pipeline is the automation path that takes an approved model artifact and promotes it safely into a serving environment. It is where ML engineering becomes operational discipline.

The goal is not only speed. The goal is repeatability, traceability, rollback capability, and separation between experimentation and production mutation.

Core Concepts

- Typical building blocks include a model registry or artifact store, automated validation, security checks, deployment targets, approval gates, and post deployment monitoring hooks.
- The pipeline should bind together model version, container image version, infrastructure configuration, and release metadata so the production state is fully reconstructable.
- Release patterns may include staging, canary, shadow, blue green, or weighted rollout depending on the serving platform and risk tolerance.

How It Works

- A training or evaluation workflow registers a candidate model and stores its metrics and artifacts in a form the deployment system can trust.
- Deployment automation validates the artifact, updates the serving configuration, shifts traffic according to the selected release strategy, and watches health and quality signals.
- If checks fail, the pipeline rolls back automatically or halts for investigation, preserving both uptime and release confidence.

AI and ML Relevance

- This project is central to production ML because a good model in a notebook has no value unless it can be released safely and repeatedly.
- It also enables governance by making promotion criteria explicit instead of relying on ad hoc approvals in chat or email.

Operational Notes

- Do not deploy from loose files or untracked notebook outputs. Promote only immutable, versioned artifacts produced by trusted automation.
- Separate technical health checks from model quality checks. A service can be up while prediction behavior is wrong.
- Rollback design should be exercised before an incident. The first rollback should not happen under panic conditions.

What To Remember

- The deployment pipeline is the contract between data science output and production reality.
- Immutable artifacts and observable rollout strategy are the backbone of safe release.
- If production cannot be traced back to a specific model version and image version, the pipeline is incomplete.

Project: Real Time Computer Vision Pipeline

07-projects/realtime-cv-pipeline

Overview

A real time computer vision pipeline handles image or video related inference under tight latency constraints. It coordinates ingestion, preprocessing, model execution, response shaping, and operational monitoring.

This kind of system is harder than a notebook demo because latency, payload size, concurrency, and failure handling all become first class engineering concerns.

Core Concepts

- Core components usually include an API layer, image transport or storage, preprocessing logic, a model runtime with GPU or optimized CPU support, and a result sink for downstream systems.
- The design must account for payload size, serialization overhead, and whether inference happens synchronously on request or asynchronously after upload.
- Observability needs to distinguish infrastructure failures from model failures, such as corrupt inputs, low confidence outputs, or drift in camera conditions.

How It Works

- A client uploads an image or frame reference, the system validates and preprocesses it, and the model runtime performs detection, classification, or segmentation.
- The service returns structured predictions quickly or stores results for later retrieval if the payload and model latency make asynchronous processing more appropriate.
- Logs, request tracing, and sampled payload metadata support debugging when quality drops or latency spikes under production traffic.

AI and ML Relevance

- This project demonstrates the gap between model training success and production reality. Input normalization, batching strategy, and hardware choice often matter as much as the model architecture.
- It is also a useful pattern for understanding how AWS services combine around GPU inference, object storage, APIs, and edge facing reliability concerns.

Operational Notes

- Test with realistic image sizes and traffic bursts. Benchmarks based on tiny local samples routinely mislead teams.
- Protect the expensive inference layer with rate limits, queues, or admission control. One malformed client can otherwise create a cost incident.
- Version preprocessing code with the model. A mismatch between training time and serving time transforms silently damages quality.

What To Remember

- Real time CV is a systems problem, not just a model problem.
- Latency budget is shared across transport, preprocessing, inference, and response handling.
- The serving path must preserve the same assumptions the model saw during training, or quality will collapse in production.

Project: Serverless ML API

07-projects/serverless-ml-api

Overview

A serverless ML API exposes model functionality through a managed API layer and event driven or on demand compute instead of continuously running servers. The design trades infrastructure simplicity against packaging and latency constraints.

It is a good pattern for intermittent usage, control plane operations around models, or lightweight inference that does not justify an always on fleet.

Core Concepts

- The typical stack is API Gateway plus Lambda, sometimes combined with S3, DynamoDB, Step Functions, or an external managed inference endpoint for heavier model execution.
- The key question is where the model runs. Very small models may execute inside Lambda, while larger models may require the API layer to dispatch to SageMaker, ECS, or an asynchronous backend.
- Security, request validation, quotas, and idempotency all sit close to the API boundary because public serverless endpoints are easy to invoke at scale.

How It Works

- A client sends a request to the API, which authenticates and validates the payload before invoking the function or orchestration path.
- The backend performs preprocessing, optional feature or state lookup, model invocation, and response shaping, then returns a result or a job reference.
- CloudWatch metrics and logs capture request count, latency, errors, and downstream dependency behavior so the system can be tuned over time.

AI and ML Relevance

- This project is useful for understanding which ML capabilities fit naturally into serverless boundaries and which do not.
- It also demonstrates how to expose prediction features with low operational overhead while preserving good access control and cost visibility.

Operational Notes

- Cold starts, package size, and downstream timeout chains are the main operational risks. Heavy frameworks or large model files can make the design fail its latency objective.
- Guard against abuse with throttling and auth because a public inference API can turn into an expensive denial of wallet event.
- Prefer asynchronous patterns when inference time or payload size approaches the practical limits of synchronous serverless execution.

What To Remember

- Serverless APIs are excellent when traffic is bursty and the unit of work is modest.
- Do not force a large real time model entirely into Lambda just because serverless looks simple on a diagram.
- The right design often mixes serverless control flow with a separate managed inference runtime.

08 Docker

Docker for ML Workloads

docker

Overview

Docker packages an application and its runtime dependencies into a portable container image. For ML work, that usually means Python dependencies, native libraries, model server processes, and sometimes CUDA aligned stacks.

Containerization matters because ML code that works on one laptop can fail elsewhere due to library versions, system packages, or GPU compatibility. Docker reduces that drift by making the runtime explicit.

Core Concepts

- A Docker image is a layered filesystem built from a Dockerfile, while a container is a running instance created from that image.
- Good ML container design separates dependency installation, application code, model loading behavior, and configuration so rebuilds stay fast and reproducible.
- Images are typically stored in a registry such as ECR and then executed by a runtime platform such as ECS, SageMaker, or Kubernetes.

How It Works

- You define a Dockerfile, build the image locally or in CI, run tests inside the image, and publish the versioned artifact to a registry.
- At deployment time, the target platform pulls the image, injects configuration and credentials, and starts the container according to the workload type.
- For inference systems, startup behavior matters because model download time, warmup steps, and dependency weight directly affect deployment speed and cold start characteristics.

AI and ML Relevance

- Docker is the standard packaging boundary for custom training jobs, model servers, feature services, and batch processing components in modern ML platforms.
- It also makes local reproduction easier because the same image can be tested before it is pushed into a managed AWS runtime.

Operational Notes

- Keep images small and deterministic. Bloated images increase build time, push time, scan time, and startup latency.
- Pin dependency versions, scan for vulnerabilities, and separate secrets from the image itself. Secrets belong in runtime configuration, not baked into layers.

- Be especially careful with GPU stacks. CUDA, driver expectations, and framework versions need to line up with the runtime environment.

What To Remember

- Docker solves packaging and runtime consistency, not orchestration or model governance.
- A reproducible image is a prerequisite for reliable ML deployment, not the whole deployment story.
- If the image cannot be rebuilt from source with the same dependencies, release confidence is weaker than it looks.

09 Terraform

Terraform for AWS ML Infrastructure

terraform

Overview

Terraform is an infrastructure as code tool that describes cloud resources declaratively and converges the real environment toward that desired state. It is useful for making AWS environments reproducible, reviewable, and less dependent on console driven manual setup.

In an ML platform, Terraform helps stabilize the foundation around networking, IAM roles, storage, compute platforms, and deployment targets so experiments are not blocked by inconsistent environments.

Core Concepts

- Terraform configurations declare resources, variables, outputs, providers, and modules. State tracks what Terraform believes exists so it can plan changes safely.
- Modules are important because ML platforms repeat patterns such as VPC layouts, endpoint roles, bucket policies, and monitoring stacks across environments.
- The key workflow is plan and apply. Teams review proposed infrastructure diffs before mutating shared environments.

How It Works

- Engineers define infrastructure in code, initialize the provider, generate a plan, and review what resources will be created, updated, or destroyed.
- After approval, Terraform applies the changes and updates state so future plans compare against the latest known environment.
- CI and CD systems can validate formatting, policy rules, and plans before production changes are allowed to execute.

AI and ML Relevance

- Terraform is valuable when ML systems span many AWS services and environments because manual provisioning makes reproducibility and compliance difficult.
- It also helps connect model deployment with the surrounding infrastructure, such as endpoint security groups, alarms, IAM roles, and data access boundaries.

Operational Notes

- Protect Terraform state because it can contain sensitive identifiers and represents the source of truth for change planning.
- Prefer modules and naming conventions over copy paste infrastructure. Drift multiplies quickly when environments are hand assembled.

- Be cautious with destructive changes and review plans carefully. Infrastructure mistakes can impact data access, uptime, and cost at large scale.

What To Remember

- Terraform makes infrastructure reviewable in the same way source code is reviewable.
- The plan output is where design intent meets operational reality. Read it carefully before applying.
- For ML teams, infrastructure as code is what turns one successful environment into a repeatable platform.

10 Certifications

Certification: AWS Machine Learning Specialty

certifications/aws-ml-specialty

Overview

The AWS Machine Learning Specialty certification is intended to test practical knowledge across the machine learning lifecycle on AWS, not just isolated service definitions. A good preparation strategy connects data, modeling, deployment, and operational reasoning.

The exam is valuable because it pushes you to understand service fit, tradeoffs, and failure modes across an end to end ML platform rather than memorizing only marketing descriptions.

Core Concepts

- Preparation should cover data engineering patterns, supervised and unsupervised model concepts, feature processing, training and tuning workflows, deployment options, monitoring, and security.
- You need both AWS specific knowledge and general ML judgment. Questions often hinge on choosing the best operational approach, not on recalling one service name in isolation.
- Hands on exposure matters. Services such as SageMaker, Glue, Athena, IAM, S3, and monitoring tools make more sense after you have seen their integration points directly.

How It Works

- Build a study loop around reading, small labs, recall practice, and architecture comparison. Passive reading alone is rarely enough.
- As you study, tie each service back to an ML stage: ingestion, preparation, training, deployment, security, cost control, or observability.
- Use practice questions to identify weak links, then return to first principles rather than memorizing the answer key mechanically.

AI and ML Relevance

- This certification aligns well with platform oriented ML roles because it emphasizes how models become production systems on AWS.
- It also helps sharpen service selection thinking, which is one of the most important practical skills when designing ML workloads under budget and compliance constraints.

Operational Notes

- Do not overfocus on one service family. Many exam questions are about choosing between multiple valid looking options under a specific business constraint.
- Anchor your preparation in tradeoffs: latency versus throughput, managed convenience versus control, batch versus real time, and security versus operational flexibility.

- Keep a personal summary sheet of patterns and anti patterns. Review architecture decisions, not only terminology.

What To Remember

- The exam rewards connected understanding more than isolated definitions.
- Know why a service fits a workload, not only what the service claims to do.
- Good preparation looks like system design practice with AWS ML components, not just rote flashcards.

Certification: AWS Solutions Architect

certifications/aws-solutions-architect

Overview

The AWS Solutions Architect path focuses on designing resilient, secure, cost aware systems on AWS. Even for an ML focused engineer, that mindset is important because production ML inherits all the usual architecture concerns of any serious cloud system.

Studying for this certification strengthens the foundation that supports data and ML workloads: networking, storage, compute, identity, resilience, and multi service design tradeoffs.

Core Concepts

- The core lens is the AWS Well Architected style of thinking: operational excellence, security, reliability, performance efficiency, cost optimization, and sustainability.
- You should be comfortable comparing compute options, database models, networking patterns, disaster recovery strategies, and identity designs across common workloads.
- Unlike an ML specific exam, this path broadens your ability to reason about the surrounding system that hosts, secures, and scales the model lifecycle.

How It Works

- Study by taking ordinary application and platform scenarios and asking which AWS services fit best under explicit constraints such as global reach, low latency, regulated data, or spiky traffic.
- Use labs to reinforce VPC networking, IAM, storage design, load balancing, autoscaling, messaging, and database selection because those building blocks appear repeatedly.
- Practice comparing architectures, not just naming services. The exam frequently tests the best answer among plausible options rather than a single obvious fact.

AI and ML Relevance

- For ML practitioners, this certification improves the surrounding engineering judgment needed for secure data access, release automation, network isolation, monitoring, and cost control.
- It is especially helpful when your role extends beyond model code into broader platform or product architecture responsibilities.

Operational Notes

- Do not neglect fundamentals because they feel less exciting than ML topics. Most production incidents arise from basic architecture weaknesses, not from exotic modeling tricks.
- Tie every service to a design tradeoff and a failure mode. Knowing when not to use something is often as important as knowing when to use it.
- Revisit cost, security, and resilience questions repeatedly. Those themes cut across nearly every architecture decision.

What To Remember

- Strong ML systems still rely on strong cloud architecture basics.
- This certification trains decision making under constraints, which transfers directly into ML platform design.
- If you can explain the tradeoff clearly, you are studying at the right depth.